

Lab 04 — De Dockerfile: je eigen images bouwen

Theorie — het recept, de context, de cache, de twee valkuilen die het meest kosten, en wat een build-engine zonder daemon verandert.

Doelstellingen

- Begrijpen wat de **build context** is en waarom die bepaalt wat je kunt kopiëren.
- De essentiële instructies kennen en wat elke ervan produceert.
- `CMD` van `ENTRYPOINT` onderscheiden, *shell*-vorm van *exec*-vorm.
- `ARG` van `ENV` onderscheiden.
- Een Dockerfile ordenen om de **buildcache** te benutten.

1. De build context

```
podman build -t mijn-api:1.0 .
```

De `.` op het einde is niet decoratief: het is de **build context**, de map die de build mag lezen. Twee absolute gevolgen:

- **Je kunt alleen kopiëren wat in de context zit.** `COPY ../secrets/sleutel.pem .` faalt altijd: het bestand ligt buiten de perimeter. Geen omweg mogelijk.
- **De hele map maakt deel uit van de context**, inclusief `.git/`, `node_modules/`, `target/`, logs en lokale configuratie. Een `COPY . .` stopt dat allemaal in de image.

PODMAN

Bij Docker **verpakt** de client de context in een archief en **stuurt** hij het naar de daemon — dat is de regel `transferring context: 900MB` die Angular-builds zo lang laat duren. Bij Podman wordt de build gedaan door **Buildah**, geïntegreerd in hetzelfde proces, dat de map rechtstreeks leest: geen archief, geen overdracht. De context blijft wel de grens van wat kopieerbaar is, en `.dockerignore` blijft onmisbaar — niet voor de snelheid, maar voor wat **in de image** terechtkomt. Buildah aanvaardt ook de neutrale namen `Containerfile` en `.containerignore`.

Het bestand `.dockerignore`, in de root van de context, sluit uit wat er niet in mag:

```
.git
node_modules
target
.env
```

VALKUIL

Zonder `.dockerignore` stopt een `COPY . .` lokale geheimen en de volledige `.git` **in de uiteindelijke image**, voor iedereen die ze downloadt. Een klassiek datalek.

De Dockerfile zelf mag elders staan: `-f docker/api.Dockerfile` wijst hem aan.

2. De essentiële instructies

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine # basisimage – altijd de 1e instructie
LABEL org.opencontainers.image.source="https://git.mijnbedrijf.be/betalingen/api"
WORKDIR /app # maakt de map aan en gaat erin
COPY target/api.jar /app/api.jar # kopieert van de context naar de image
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75" # variabele aanwezig bij uitvoering
EXPOSE 8080 # documentatie: publiceert GEEN poort
USER 1000:1000 # niet als root draaien
ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]
```

Instructie	Rol	Bestandslaag?
FROM	Vertrekpunt	ja (die van de basis)
RUN	Voert een commando uit bij de build	ja
COPY / ADD	Kopieert vanuit de context	ja
WORKDIR, ENV, USER, EXPOSE, LABEL	Metadata	nee (0B)
CMD, ENTRYPOINT	Wat uitgevoerd wordt bij <code>run</code>	nee
ARG	Variabele alleen voor de build	nee

Drie verduidelijkingen. **COPY eerder dan ADD**: `ADD` pakt archieven uit en downloadt URL's, twee impliciete gedragingen. **EXPOSE publiceert niets**: `-p` publiceert (lab 07). **RUN wordt uitgevoerd bij de build**: `RUN java -jar api.jar` zou de applicatie starten tijdens de constructie.

ONTHOUDEN

Schrijf de `FROM` voluit: `docker.io/library/eclipse-temurin:21-jre-alpine`. Een `FROM eclipse-temurin:...` hangt af van de configuratie van de bouwmachine (lab 02); in een bedrijf wordt het `registry.intern/basis/...`.

3. CMD tegenover ENTRYPOINT

Beide bepalen wat er bij het opstarten draait. Hun verschil is hun verhouding tot de argumenten van `podman run`:

- CMD** is een **standaardwaarde, vervangbaar**. `podman run mijn-image ander-commando` negeert de `CMD`.
- ENTRYPOINT** is het **vaste** programma. De argumenten van `podman run` worden eraan **toegevoegd**.

```
ENTRYPOINT ["java","-jar","/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

`podman run api` start `java -jar /app/api.jar --spring.profiles.active=prod`; `podman run api --spring.profiles.active=dev` start hetzelfde met het profiel `dev`. Dat is het standaardpatroon: **ENTRYPOINT** legt het programma vast, **CMD** levert de standaardargumenten; `podman run --entrypoint sh -it mijn-image` blijft de nooduitgang om te debuggen.

→ SPRING BOOT

Argumenten na de JAR (`--spring.profiles.active=dev` , `--server.port=9090`) leest Spring als eigenschappen die voorrang hebben op `application.yml` . Vandaar het gemak van het duo `ENTRYPOINT` + `CMD` : dezelfde image, een ander argument per omgeving. Lab 08 toont dat omgevingsvariabelen nog beter zijn.

4. Shell-vorm en exec-vorm

Elk commando kan op twee manieren geschreven worden, en dat is geen kwestie van stijl:

```
CMD java -jar /app/api.jar # SHELL-vorm -> /bin/sh -c "java -jar ..."  
CMD ["java","-jar","/app/api.jar"] # EXEC-vorm -> java wordt PID 1
```

In exec-vorm **is** de applicatie PID 1: ze ontvangt `SIGTERM` en stopt netjes. In *shell*-vorm schuift er een `/bin/sh` tussen — het probleem van lab 03: een shell die PID 1 blijft, geeft `SIGTERM` niet door, de applicatie krijgt het nooit, `stop` wacht tien seconden en doodt alles.

Het belangrijke woord is **blijft**. Het gedrag hangt af van de shell-implementatie:

Geval	PID 1	podman stop
Exec-vorm	je applicatie	netjes, code 143
Shell-vorm, eenvoudig commando, basis Alpine (busybox)	je applicatie (de shell stapt opzij)	netjes, code 143
Shell-vorm, eenvoudig commando, basis Debian/Ubuntu (dash)	<code>/bin/sh</code>	10 s en dan code 137
Shell-vorm met een pipe, een <code>&</code> , een <code>;</code>	<code>/bin/sh</code>	10 s en dan code 137
Opstartscript dat de app start zonder <code>exec</code>	<code>/bin/sh</code>	10 s en dan code 137

→ LINUX / SHELL

`/bin/sh` is geen uniek programma: `dash` op Debian en Ubuntu, `ash` van busybox op Alpine. Sommige vervangen zichzelf door het commando wanneer dat het *laatste* van het script is (een impliciete `exec`); andere maken een kind aan en wachten. Vandaar een Dockerfile die netjes stopt op Alpine en niet op Debian.

ONTHOUDEN

Schrijf altijd de exec-vorm, met JSON-dubbele aanhalingstekens. Moet je door een shell, schrijf die dan expliciet **en** gebruik `exec` : `ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]` .

Een minder bekend gevolg: in exec-vorm worden `$JAVA_OPTS` , `&&` , `|` en `>` **niet** geïnterpreteerd — er is geen shell om dat te doen.

5. ARG tegenover ENV

```
ARG VERSION=1.0 # alleen beschikbaar tijdens de build  
ENV APP_VERSION=${VERSION} # blijft in de image en in de containers
```

	ARG	ENV
Zichtbaar tijdens de build	ja	ja
Aanwezig in de uiteindelijke image	nee	ja
Wijzigbaar bij de build	<code>--build-arg VERSION=2.0</code>	nee
Wijzigbaar bij uitvoering	nee	<code>podman run -e APP_VERSION=...</code>

VALKUIL

"ARG verdwijnt uit de image" betekent **niet** "ARG is veilig voor een geheim": de waarde blijft zichtbaar in `podman history` en in de buildcache. Een wachtwoord via `--build-arg` is een lek (lab 08).

6. De buildcache

De engine verwerkt de instructies in volgorde en cachet elk resultaat. Voor elke instructie vraagt hij zich af: "heb ik deze al uitgevoerd, vanuit dezelfde vorige laag?" Zo ja, dan hergebruikt hij (`--> Using cache`). Zo niet, dan voert hij ze uit — **en maakt hij alles wat volgt ongeldig**. Ongeldigmaking komt van een wijziging in de tekst van de instructie, in de **inhoud** van gekopieerde bestanden (`COPY` / `ADD`), of van een vorige ongeldig gemaakte instructie, in cascade. Vandaar de gulden regel: **van het meest stabiele naar het meest vluchtige**.

```
# SLECHT: de code verandert bij elke commit, dus alles wordt herbouwd
COPY . /app
RUN mvn dependency:go-offline

# GOED: de afhankelijkheden worden alleen opnieuw gedownload als pom.xml verandert
COPY pom.xml /app/
RUN mvn dependency:go-offline
COPY src /app/src
```

Dezelfde redenering voor Angular: `COPY package*.json` en dan `npm ci`, en pas daarna `COPY . .`. De winst telt in minuten per CI-build — en, aangezien een ongewijzigde laag niet opnieuw overgedragen wordt bij `push` (lab 02), in uitroltijd. Om alles te herbouwen: `--no-cache`.

7. Een correcte RUN schrijven

```
# SLECHT: drie lagen, een apt-cache van 40 MB meegeleverd in de image
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOED: één enkele laag, effectieve opruiming
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
```

De opruiming moet **in dezelfde RUN** gebeuren als de installatie: een latere verwijdering verbergt zonder weg te nemen (lab 02). En `apt-get update` mag nooit alleen in zijn laag staan: wekenlang

gecacht zou het verouderde indexen serveren — het probleem van *cache busting*.

8. In het bedrijf

De Dockerfile van een Spring Boot-backend ziet er zo uit (eenvoudige versie; de multi-stage-versie komt in lab 05):

```
FROM registry.intern/basis/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api-*.jar app.jar
EXPOSE 8080
USER 1000:1000
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Merk op: een **JRE**-image en geen JDK, gehaald uit de interne registry, niet-root `USER`, exec-vorm, JAR gekopieerd uit `target/` — de Maven-build gebeurde dus **vooraf**, in de CI; dat is de beperking die multi-stage zal opheffen. Aan Angular-zijde containeriseer je nooit `ng serve`, maar het resultaat van `ng build`, geserveerd door nginx. Hetzelfde recept wordt door Docker in de CI en door Podman op je werkpost gebouwd: een Dockerfile is een standaard.

Onthouden

- De `.` van `build .` wijst de **context** aan: wat kopieerbaar is, en wat een `COPY . .` meeneemt. `.dockerignore` is verplicht — ook zonder overdracht, met Podman.
- `EXPOSE` documenteert, `-p` publiceert. `RUN` draait bij de build, `CMD` / `ENTRYPOINT` bij de uitvoering; `ENTRYPOINT` legt het programma vast, `CMD` levert vervangbare argumenten.
- Exec-vorm `["prog", "arg"]`: de applicatie is PID 1 en ontvangt `SIGTERM`. *Shell*-vorm: hangt af van de shell van de basisimage — dus nee. `ARG` leeft tijdens de build, `ENV` blijft in de image — geen van beide is geschikt voor een geheim.
- Orden van het meest stabiele naar het meest vluchtige; elke ongeldig gemaakte instructie maakt de volgende ongeldig.
- Installatie en opruiming in **dezelfde** `RUN`, anders blijven de bestanden.

Woordenschat

build context: map die de build mag lezen. — `.dockerignore` / `.containerignore`: uitsluitingen uit de context. — **Containerfile**: neutrale naam van de Dockerfile bij Podman. — **Buildah**: build-engine van Podman. — **exec-/shell-vorm**: twee schrijfwijzen van `CMD` / `ENTRYPOINT`. — **cache busting**: bewuste ongeldigmaking van de cache. — **basisimage**: image genoemd door `FROM`.